
Chapitre 1

Cadre général du projet

Plan :

Introduction.....	3
I. Cadre de projet	3
II. tude de l'existant.....	4
III. Objectifs du projet.....	6
IV. Méthodologie adoptée.....	6
Conclusion	11

Introduction

Ce premier chapitre a pour objectif de mettre le projet dans son cadre général. Nous commençons par la présentation de l'organisme d'accueil. Par la suite nous présentons notre projet ainsi que l'étude et la critique de l'existant. Finalement, nous spécifions la solution proposée et les objectifs visés et aussi la méthodologie de réalisation adoptée.

I. Cadre de projet

Le travail présenté dans ce rapport a été effectué dans le cadre d'un projet de fin d'études en vue de l'obtention de fin d'études en vue de l'obtention du diplôme national de licence en Business Computing à l'Institut Supérieur d'Administration des Entreprises de Gafsa (ISAEG).

❖ **Présentation du projet**

Ce projet consiste à concevoir et développer une plateforme web intelligente de gestion d'un cabinet dentaire. Cette plateforme vise à digitaliser l'ensemble des processus liés à la gestion des patients, des rendez-vous, des dossiers médicaux.

II. Étude de l'existant

L'étude de l'existant permet d'analyser les solutions d'e-learning actuelles et leurs limites.

Elle aide à identifier les besoins réels des enseignants, des élèves et des parents afin de concevoir une plateforme plus efficace et mieux adaptée au suivi pédagogique.

1. Analyse de l'existant

Actuellement, de nombreux cabinets dentaires utilisent encore des méthodes traditionnelles pour gérer leur activité. Les dossiers patients sont souvent stockés sous format papier ou dans des fichiers numériques non centralisés. La prise de rendez-vous se fait généralement par téléphone ou sur place, ce qui entraîne une charge administrative importante.

Les outils existants se limitent généralement à de simples logiciels de facturation ou à des agendas numériques basiques, sans réelle intégration entre les différents aspects médicaux, administratifs et analytiques.

2. Critique de l'existant

- L'analyse de l'existant met en évidence plusieurs limites majeures :
- Gestion administrative lourde et manuelle.
- Risque de perte ou de dégradation des dossiers médicaux
- Difficulté de suivi de l'historique médical des patients.
- Mauvaise optimisation du planning du dentiste.
- Absence de rappels automatiques, entraînant des absences fréquentes.
- Manque d'outils d'aide à la décision et d'analyse.
- Communication limitée entre le cabinet et les patients.
- Ces contraintes impactent négativement la productivité du personnel, la qualité du suivi médical et l'expérience globale du patient..

3. Solution proposée

Pour répondre à ces problématiques, nous proposons le développement d'une plateforme web intelligente qui centralise l'ensemble des données du cabinet dentaire. Cette solution offre une gestion numérique complète des patients ainsi que de leurs dossiers médicaux, facilitant ainsi le suivi et l'organisation des consultations. Elle intègre également un système de prise de rendez-vous en ligne, accompagné de rappels automatiques pour améliorer la ponctualité et la communication avec les patients. De plus, la plateforme permet la génération numérique des ordonnances et devis, simplifiant les démarches administratives. Enfin, un tableau de bord analytique est mis à disposition du dentiste, offrant une vision claire et synthétique de l'activité du cabinet pour une meilleure prise de décision.

III. Objectifs du projet :

L'objectif principal de ce projet est de développer une plateforme web performante et sécurisée dédiée à la gestion intelligente d'un cabinet dentaire. Cette plateforme vise à digitaliser l'ensemble des processus administratifs et médicaux afin de réduire les tâches répétitives du personnel, d'améliorer la qualité du suivi médical des patients et d'optimiser la planification des rendez-vous. Elle permet également d'améliorer l'expérience globale du patient grâce à des services numériques accessibles et efficaces, tout en fournissant au dentiste et au personnel du cabinet des indicateurs clairs et pertinents pour faciliter la prise de décision et le pilotage de l'activité.

IV. Méthodologie adoptée

1. Méthodes agiles

Le cycle de vie d'un logiciel, désigne toutes les étapes du développement d'un logiciel, dès sa conception jusqu' à sa disparition. L'objectif d'un tel découpage est de permettre de définir des jalons intermédiaires permettant la validation du développement logiciel, c'est-à-dire la conformité du logiciel avec les besoins exprimés, et la vérification du processus de développement, c'est-à-dire l'adéquation des méthodes mises en œuvre. Pour chaque produit (ou application) conçu et développé nous choisissons une démarche pour la suivre tout au long du projet que l'on appelle cycle de vie du produit. Il existe plusieurs types de cycles de vie notamment Cycle de vie en V, en cascade, spirale [1].

Le concept de méthodologie vient remplacer le cycle de vie et l'enrichit par une démarche organisationnelle du projet. La méthodologie permet ainsi de structurer, planifier et contrôler le processus de développement. Pour arriver à faire un choix judicieux de la méthodologie à entreprendre, nous proposons de mener une étude comparative sur les méthodologies existantes

Suite à cette étude comparative, nous avons dégagé le tableau 1 qui présente la différence entre les méthodes classiques tels que cascade, Spirale et les méthodes agiles (Scrum, XP RUP).

Tableau 1: Etude comparative entre Approche traditionnelle et Approche Agile

Thème	Approches traditionnelles	Approches Agile
Cycle de vie	En cascade ou en V, sans rétroaction possible, phases séquentielles.	Itératif et incrémental
Planification	Prédictive, caractérisée par des plans plus ou moins détaillés sur la base d'un périmètre et d'exigences définies et stables au début du projet.	Adaptative avec plusieurs niveaux de planification macro et micro planification avec ajustements si nécessaires au fil de l'eau en fonction des changements survenus.
Documentation	Produite en quantité importante comme support de communication, de validation et contractualisation.	Réduite au strict au profit d'incrément fonctionnels opérationnels pour obtenir le feedback du client.
Equipe	Une équipe avec des ressources spécialisées, dirigées par un chef de projet.	Une équipe responsable où l'initiative de la communication est privilégiée et soutenue par le chef de projet.
Qualité	Contrôle qualité à la fin Du cycle de développement. Le client découvre le produit fini.	Un contrôle qualité est permanent, au niveau du produit et des ressources. Le client visualise les résultats tôt et fréquemment.
Changement	Résistance voire opposition au changement processus lourd de gestion des changements acceptés.	Accueil favorable au changement inéluctable, intégré dans le processus.
Suivi de l'avancement	Mesure de la conformité aux plans initiaux. Analyse des écarts	Un seul indicateur d'avancement : le nombre de fonctionnalités implémentées et le travail restant à faire.
Gestion des risques	Processus distinct, rigoureux, de gestion des risques.	Gestion des risques intégrée dans le processus global, avec Responsabilisation de chacun dans l'identification et la résolution des risques. Pilotage par les risques.
Mesure succès	Respect des engagements initiaux en termes de cout, De budget et de niveau de qualité.	Satisfaction client par la livraison de valeur ajoutée.

1.1 Présentation de SCRUM

« Scrum » est une approche de gestion de projets Agile qui vise à améliorer la productivité des équipes agiles, même lorsqu'elles travaillent à distance. Son objectif principal est d'optimiser le produit en facilitant des feedbacks réguliers avec les utilisateurs finaux. La méthode « Scrum » tire son appellation du domaine du rugby, où le terme « Scrum » fait référence à une mêlée. De manière similaire, les équipes agiles qui adoptent « Scrum » se réunissent régulièrement pour s'assurer que le projet progresse de manière appropriée, prêtes à le réajuster en cours de route. Cela crée une approche dynamique et collaborative de la gestion du projet, offrant au client un équilibre optimal entre l'investissement initial et le produit final livré. « Scrum » est actuellement l'approche agile la plus appréciée par les équipes de développeurs, car elle met en avant les valeurs fondamentales de l'Agile : la collaboration avec le client, l'acceptation du changement, l'interaction avec les individus et la livraison de logiciels opérationnels. « Scrum » fait partie des méthodes agiles les plus offrant également la possibilité de mettre à l'échelle, c'est-à-dire de déployer progressivement l'agilité à l'échelle de l'entreprise. [2]

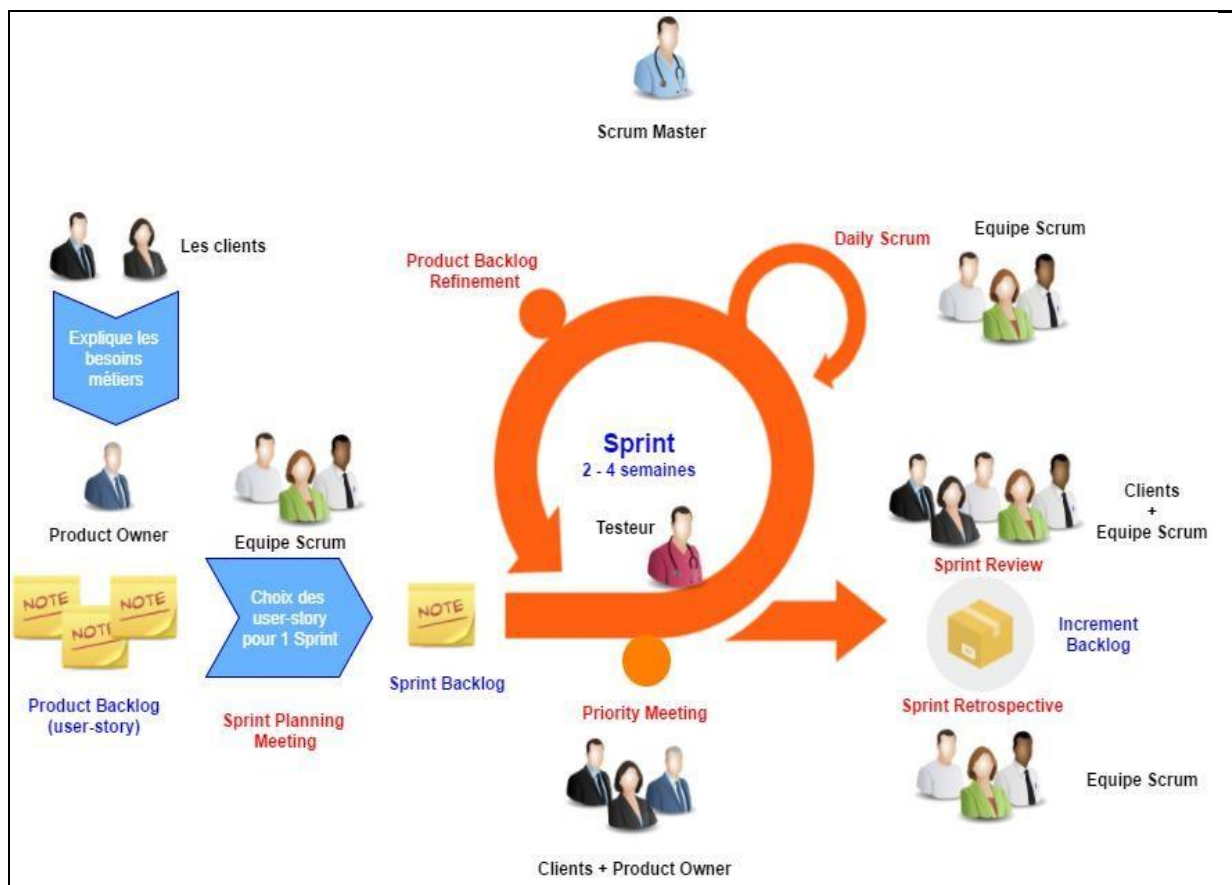


Figure : Cycle de vie en SCRUM

1.2 Les rôles SCRUM

Les membres d'une équipe SCRUM partagent leur temps entre trois rôles « Scrum » différents sans aucune relation hiérarchique entre eux.

1.2.1 Le Scrum Master

- Assure les principes et les valeurs de « Scrum » sont respectées.
- Facilite la communication au sein de l'équipe.
- Cherche à améliorer la productivité et le savoir-faire de son équipe.

1.2.2 L'équipe

- Tous les membres de l'équipe apportent leur savoir-faire pour accomplir les tâches.
- Taille de 3 à 9 personnes en général.
- Pas de rôle bien déterminé : architecte, développeur, testeur.

1.2.3 Le Product Owner

- Expert métier, définit les spécifications fonctionnelles.
- Etablit la priorité des fonctionnalités à développées ou a corrigées.
- Valide les fonctionnalités développées.
- Joue le rôle du client.

1.3 Les outils Scrum

1.3.1 Le Scrum board

Il s'agit d'un outil utilisé dans la méthode « Scrum » qui prend la forme d'un tableau permettant de suivre en temps réel l'avancement des tâches et des « user stories » à accomplir. Cet outil est divisé en au moins trois sections : les tâches à effectuer, les tâches en cours et les tâches terminées. Grâce à cet outil, il est possible de visualiser clairement l'état d'avancement du projet et de mieux coordonner les activités de l'équipe.

1.3.2 Le Scrum burndown chart

Il s'agit d'un graphique qui permet de se positionner quotidiennement en termes de progression et de charge de travail restante par rapport aux prévisions initiales. Cet outil visuel offre une représentation claire de l'état d'avancement du projet et permet de surveiller de près l'écart par rapport aux estimations initiales. Il facilite ainsi le suivi de la performance et l'ajustement des ressources en fonction de l'évolution réelle du travail.

1.3.3 Le Backlog

Il s'agit d'une liste des fonctionnalités attendues pour un produit. En plus des aspects fonctionnels, cette liste inclut tous les éléments qui nécessiteront du travail de la part de l'équipe. Les éléments sont classés par ordre de priorité, ce qui permet de définir l'ordre de réalisation. Cette liste joue un rôle crucial dans la planification et la gestion des tâches, en assurant que les fonctionnalités les plus importantes sont traitées en premier.

2. Le langage de conception UML

Dans un projet « Scrum », la phase de conception implique l'utilisation de l'outil de modélisation visuelle appelé langage de modélisation unifié (UML). Les diagrammes UML permettent de représenter visuellement les différents aspects d'un système tel que les 12 relations, le comportement, la structure et les fonctionnalités. Ce langage de modélisation propose 14 types de diagrammes différents, notamment :

2.1 Diagramme de cas d'utilisation :

Le diagramme de cas d'utilisation illustre la manière dont les fonctionnalités requises par les utilisateurs du système sont organisées. Chaque cas d'utilisation représente une interaction distincte entre un utilisateur, appelé acteur, et le système, qui s'établit par l'envoi ou la réception de messages.

- Les acteurs : Un acteur peut prendre la forme d'une personne, d'une organisation ou d'un système externe qui interagit avec le système en question. Il représente un élément externe qui participe à l'échange de données, que ce soit en les produisant ou en les consommant.
- Le système : Une séquence spécifique d'actions et d'interactions entre les acteurs et le système se déroule. [3]

2.2 Diagramme de classe

Le diagramme de classe est une représentation graphique largement utilisée en génie logiciel pour décrire une classe, qui représente un objet ou un groupe d'objets partageant des caractéristiques et des comportements communs. Les diagrammes de classe sont extrêmement utiles à différentes étapes de la conception d'un système, notamment lors de l'analyse.

- Objet : Entité concrète ou abstraite du domaine d'application.
- Attributs : Caractéristique partagée par tous les objets de la classe.
- Opérations : Service qui peut être demandé à tout objet de la classe.
- Association entre classes : Relation binaire (en général). [4]

2.3 Diagramme de séquence

Les diagrammes de séquences sont utilisés pour décrire les interactions entre les différents éléments du système ainsi qu'avec les acteurs impliqués.

- Une ligne de vie : représente un participant à une interaction (objet ou acteur).
- Les messages : L'opération et éventuellement ses paramètres
- Quelques Fragments combinés : Alt : conditionnelle, Loop : boucle, Ref : référence à un autre diagramme de séquence. [5]

Conclusion

Dans ce chapitre introductif, nous avons présenté l'organisme d'accueil, l'étude existante, son analyse et sa critique. Nous avons ensuite exposé la solution proposée, les étapes de sa mise en œuvre ainsi que la méthodologie adoptée. Le chapitre suivant sera dédié à l'expression des besoins et à l'étude technique.

Chapitre 2

Sprint 0 Initiation du projet

Plan :

Introduction.....	13
I. Analyse et spécification des besoins.....	13
II. Pilotage du projet avec Scrum.....	15
III. Environnement langages et outils.....	22
V. Architecture générale de l'application.....	25
Conclusion	26

Introduction

Ce chapitre présente le sprint zéro, qui représente la première étape dans la réalisation du projet. Dans un premier temps, nous allons identifier les acteurs de notre plateforme ainsi que les besoins fonctionnels et non fonctionnels. Finalement, ce sprint présentera les différents environnements de travail et l'architecture générale du projet.

I. Analyse et spécification des besoins

1. Définition de l'acteur

Un acteur est une entité qui définit le rôle joué par un utilisateur ou par un système qui interagit un acteur avec le système modélisé.

Dans notre application on a 3 acteurs qui sont :

- **Le dentiste** : s'authentifie afin d'accéder à l'espace médical sécurisé de la plateforme. Il consulte la liste des rendez-vous planifiés, gère les consultations et accède aux dossiers médicaux des patients. Il est chargé de rédiger et d'attribuer les ordonnances numériques. Le dentiste dispose également d'un tableau de bord lui permettant de visualiser des statistiques liées à son activité, telles que le nombre de rendez-vous ou l'état général du planning.
- **L'assistant(e) du cabinet** : s'authentifie pour accéder aux fonctionnalités administratives. Il ou elle gère les demandes de rendez-vous en les approuvant ou en les rejetant, et propose des dates alternatives lorsque aucun créneau n'est disponible. L'assistant(e) organise et ajuste le planning du dentiste afin d'assurer une gestion optimale des rendez-vous et du flux des patients..
- **Le patient** : crée un compte et s'authentifie pour accéder à son espace personnel. Il peut effectuer des demandes de rendez-vous en ligne, annuler ou refuser un rendez-vous, et consulter l'historique de ses consultations. Le patient a également la possibilité d'accéder à ses ordonnances numériques et de visualiser sa position dans la liste d'attente, ce qui améliore la transparence et la communication avec le cabinet.

2. Spécification des besoins

2.1 Besoins fonctionnels

Le tableau ci-dessous présente les rôles de chacun des acteurs identifiés.

Tableau : Besoins fonctionnels

Acteur	Rôle / Fonctionnalités
Dentiste	• S'authentifier pour accéder à la plateforme. • Consulter la liste des rendez-vous planifiés • Gérer et attribuer les ordonnances aux patients. • Accéder à un tableau de bord présentant les statistiques du cabinet (nombre de rendez-vous, activité, etc.) • Changer la langue d'affichage de l'interface.
Assistant(e) du cabinet	• S'authentifier pour accéder à l'espace administratif. • Approuver ou rejeter les demandes de rendez-vous • Proposer des dates alternatives lorsque aucun créneau n'est disponible • Organiser et ajuster le planning des rendez-vous. • Changer la langue de l'interface.
Patient	• Créer un compte utilisateur. • S'authentifier pour accéder à son espace personnel. • Demander et réserver un rendez-vous. • Annuler ou refuser un rendez-vous. • Consulter l'historique de ses rendez-vous. • Accéder à ses ordonnances. • Consulter sa position dans la liste d'attente. • Utiliser le chatbot IA statique pour poser des questions fréquentes. • Changer la langue d'affichage de la plateforme.

2.2 Besoins non fonctionnels

Après avoir examiné en détail les besoins fonctionnels de mon application dans la partie précédente, je vais maintenant me concentrer sur les besoins non fonctionnels qu'elle offre, qui comprennent :

- ✚ **Sécurité et fiabilité** : La protection des données utilisateur est une priorité absolue. L'application est conçue pour sécuriser toutes les informations sensibles et personnelles contre les accès non autorisés, garantissant ainsi la confidentialité et la sécurité des données des écoles, des enseignants et des parents. Des protocoles de sécurité avancés sont intégrés pour prévenir toute violation de données.
- ✚ **Ergonomie des interfaces** : Les interfaces doivent être agréable à l'œil, simple et conviviale
- ✚ **Maintenance et évolutivité** : Afin d'assurer la pérennité et l'adaptabilité de l'application face aux évolutions technologiques et aux besoins changeants des utilisateurs, un plan de maintenance rigoureux est mis en place. Cela inclut des mises à jour régulières pour améliorer les fonctionnalités existantes et en introduire de nouvelles, assurant ainsi que l'application reste à la pointe de la technologie éducative.
- ✚ **La performance** : Notre application doit fournir un temps de réponse minimum et des fonctionnalités qui satisfaites les besoins des utilisateurs.

II. Pilotage du projet avec Scrum

1. Équipe et rôle Scrum

L'équipe « Scrum » est constituée d'un propriétaire de produit, de l'équipe de développement et d'un Scrum master.

L'équipe « Scrum » qui a participé à la réalisation de ce projet se définit comme suit :

Product Owner :

Elle établit les exigences, les priorités et les fonctionnalités et gère le travail de l'équipe de développement.

Scrum Master :

Il assure l'organisation des réunions et la bonne application de la méthode « AGILE » par ce biais.

L'équipe de développement :

Elle effectue le travail nécessaire pour aboutir à une augmentation de la quantité de produit pouvant être livrée à la fin de chaque itération.

2. Backlog de produit

Le Product Backlog est l'artefact fondamental de Scrum, un ensemble de spécifications techniques, fonctionnelles ou méthodologiques qui composent le produit à réaliser. Les fonctionnalités sont appelées user stories. Le tableau suivant illustre le Product Backlog de notre application :

Tableau :Backlog du Produit

ID	Acteur	Fonctionnalité	Description	Priorité
1	Patient	Création de compte	Permettre au patient de créer un compte personnel sur la plateforme	Haute
2	Patient	Authentification	Se connecter à la plateforme via des identifiants sécurisés	Haute
3	Patient	Demande de rendez-vous	Effectuer une demande de rendez-vous en ligne	Haute
4	Patient	Annulation de rendez-vous	Annuler ou refuser un rendez-vous confirmé	Moyenne
5	Patient	Consultation de l'historique	Consulter l'historique des rendez-vous et consultations	Moyenne
6	Patient	Consultation des ordonnances	Accéder et consulter les ordonnances attribuées par le dentiste	Moyenne
7	Patient	Consultation de la liste d'attente	Consulter sa position dans la liste d'attente	Moyenne
8	Patient	Utilisation du chatbot	Poser des questions fréquentes et obtenir une assistance automatique	Basse
9	Patient	Changement de langue	Choisir la langue d'affichage de la plateforme	Basse
10	Assistant(e)	Authentification	Accéder à la plateforme via un compte sécurisé	Haute
11	Assistant(e)	Validation des rendez-vous	Approuver ou rejeter	Haute

Chapitre 2 : Sprint 0 Initiation de projet

			les demandes de rendez-vous	
12	Assistant(e)	Proposition de dates alternatives	Proposer des dates alternatives en cas d'indisponibilité	Moyenne
13	Assistant(e)	Gestion du planning	Organiser et ajuster le planning des rendez-vous	Moyenne
14	Assistant(e)	Utilisation du chatbot	Utiliser le chatbot pour assistance rapide et réponses aux questions	Basse
15	Assistant(e)	Changement de langue	Sélectionner la langue de l'interface	Basse

3. Diagrammes de cas d'utilisation globale

La figure n* présente le diagramme de cas d'utilisation global, illustrant l'ensemble des fonctionnalités offertes par la plateforme aux différents acteurs du système, à savoir le dentiste, l'assistant(e) du cabinet et le patient. Ce diagramme met en évidence les interactions principales entre les utilisateurs et l'application, ainsi que les fonctionnalités générales attendues pour assurer une gestion efficace des rendez-vous, des ordonnances et du suivi des patients. Dans ce qui suit, les cas d'utilisation les plus pertinents seront décrits en détail

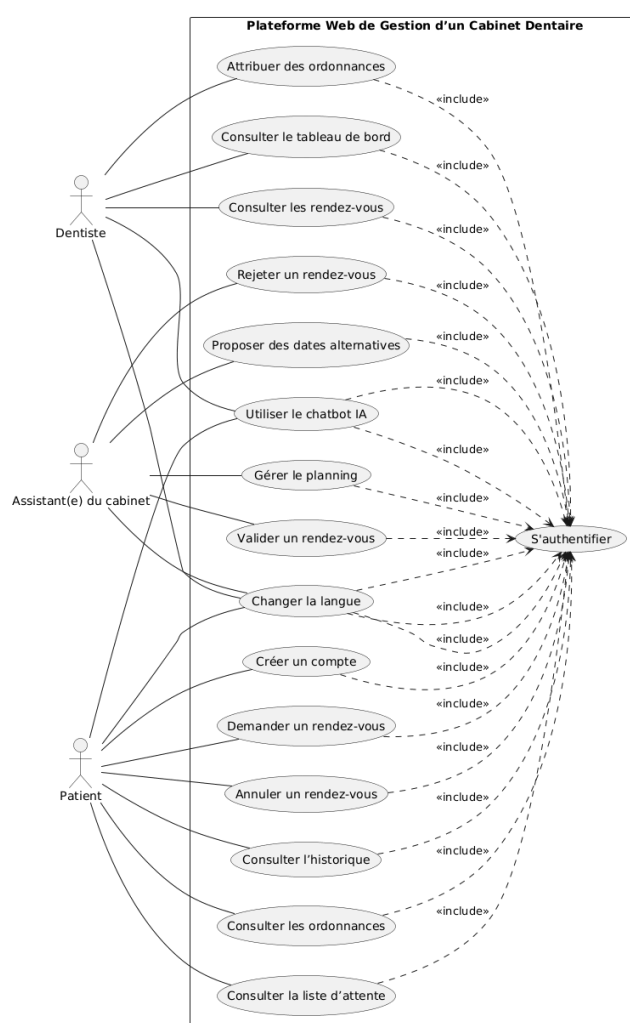


Figure 6: Diagramme de cas d'utilisation globale

4. Diagramme de conception de classe

Les diagrammes de classes sont l'un des types de diagrammes UML les plus utiles, car ils décrivent clairement la structure d'un système particulier en modélisant ses classes, ses attributs, ses opérations et les relations entre ses objets.

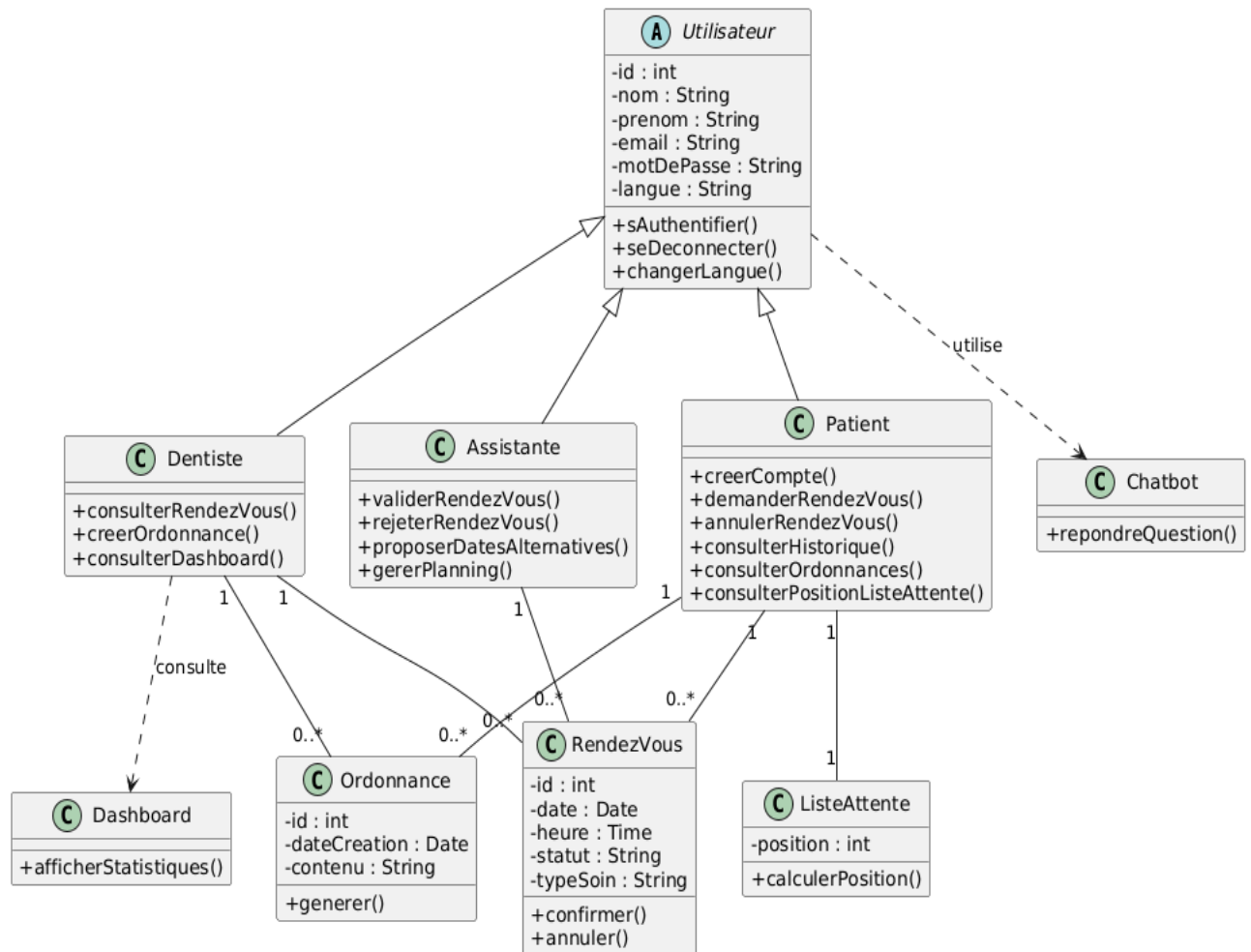


Figure : Diagramme de classe

5. Gestion de projet

5.1 Planification des sprints

Le planning de sprint est une étape clé de la méthodologie Scrum qui marque le début du sprint. Son objectif est de déterminer ce qui peut être livré lors du sprint et comment y parvenir. Cette étape consiste à collaborer étroitement avec toute l'équipe Scrum.

Tableau : Planification des sprints

Sprint	Rôle	Cas d'utilisation (Fonctionnalités)	Estimation
Sprint 1	Patient	- Création de compte - Authentification - Demande de rendez-vous - Annulation de rendez-vous - Consultation de l'historique - Consultation des ordonnances- Consultation de la liste d'attente	15 jours
Sprint 2	Assistant(e)	- Authentification - Validation des rendez-vous - Proposition de dates alternatives - Gestion du planning	15 jours
Sprint 3	Patient & Assistant(e)	- Utilisation du chatbot (Patient) - Utilisation du chatbot IA (Assistant(e)) - Changement de langue (Patient & Assistant(e))	7-10 jours

III. Environnement langages et outils

La présente section met en avant les outils de travail ayant permis de mettre en œuvre notre application et d'obtenir le produit final.

1. Les langages



Figure 10 : Logo HTML

HTML (HyperText Markup Language) : Un langage de balisage standard pour la création de pages Web. En fait, il a plusieurs cas d'utilisation tels que :

- Développement Web : concevoir la manière dont un navigateur affiche les éléments de la page Web, tels que le texte, les hyperliens et les fichiers multimédias.
- Navigation Internet : naviguer et insérer des liens entre des pages et des sites Web.
- Documentation Web : permet d'organiser et de formater des documents, de la même manière que Microsoft Word [6]



Figure 11: Logo JavaScript

JavaScript : Un langage de programmation de scripts léger, orienté objet, principalement connu comme le langage de script des pages web.[7]



Figure 12: Logo CSS

CSS : est un langage de feuille de style utilisé pour la mise en forme et la présentation des pages web. Il est utilisé pour contrôler l'apparence des éléments HTML et pour créer des mises en page complexes (developer.mozilla.org, 2023).[8]



Figure 13 : Logo Tailwindcss

Tailwindcss: est une collection d'outils utiles à la création du design (graphisme, animation et interactions avec la page dans le navigateur, etc.) de sites et d'applications web. C'est un ensemble qui contient des codes CSS



Figure 13 : Logo Bootstrap

Bootstrap : est une collection d'outils utiles à la création du design (graphisme, animation et interactions avec la page dans le navigateur, etc.) de sites et d'applications web. C'est un ensemble qui contient des codes HTML et CSS, des formulaires, des boutons et des outils de navigation.[9]

2. Les outils



Figure 15: Logo StarUML

StarUML Est un logiciel de modélisation UML (Unified Modeling Language) open source qui peuvent remplacer dans bien des situations des logiciels commerciaux et coûteux comme Rational Rose¹ ou Together². Étant simple d'utilisation, nécessitant peu de ressources système, supportant UML 2, ce logiciel constitue une excellente option pour une familiarisation à la modélisation. [11]



Figure 17: Logo Visual code

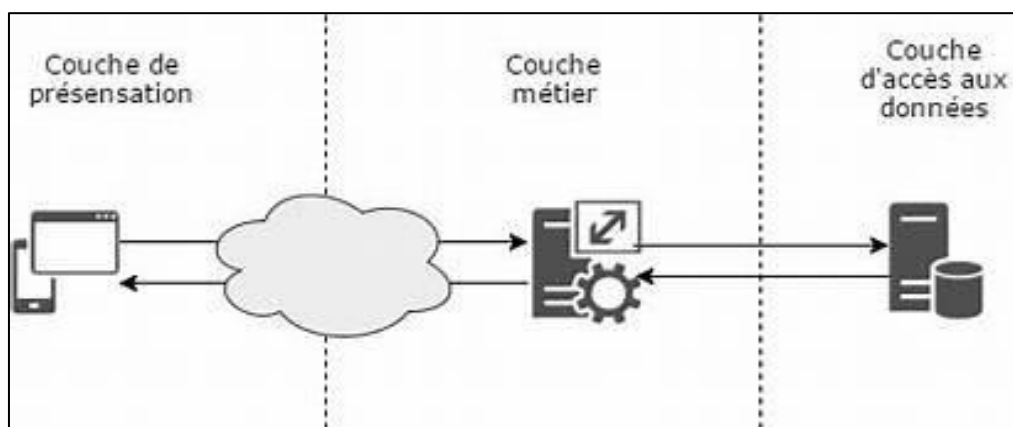
Visual code Visual Studio Code (VS Code) est un éditeur de code source développé par Microsoft. C'est un outil léger et puissant utilisé par de nombreux développeurs pour écrire, éditer et déboguer du code. Il offre des fonctionnalités avancées telles que la coloration syntaxique, l'autocomplétions intelligente, le débogage intégré, le contrôle de version, et la personnalisation grâce à une large gamme d'extensions.



Figure 18: Logo Supabsae

Postgress (Supabase) : Supabase est une plateforme open source qui fournit une base de données PostgreSQL hébergée, accompagnée d'outils backend pour le développement d'applications web et mobiles. Elle offre des fonctionnalités clés telles que l'authentification et le stockage de fichiers.

V. Architecture générale de l'application



L'architecture adoptée est basée sur l'architecture « **MVC** », qui signifie Model-View Controller. Il s'agit d'un modèle architectural qui divise une application en trois composants principaux : les modèles, les vues et les contrôleurs, chacun étant un aspect de développement spécifique d'une application conçue pour être gérée.

Figure 19: Architecture physique de l'application

1. Les différentes composantes de l'architecture MVC

- **Modèle :** Cette section est responsable de la gestion des données. Son objectif est de récupérer les informations "brutes" à partir de la base de données, de les structurer et de les regrouper de manière qu'elles puissent être traitées ultérieurement par le responsable du traitement.
- **Vue :** Cette section est principalement dédiée à l'affichage. Elle se focalise sur la récupération de variables pour déterminer les éléments à afficher, sans effectuer de calculs significatifs.
- **Contrôleur :** Cette partie est chargée de la logique du code qui effectue des prises de décision. Elle joue un rôle d'intermédiaire essentiel entre le modèle et la vue.

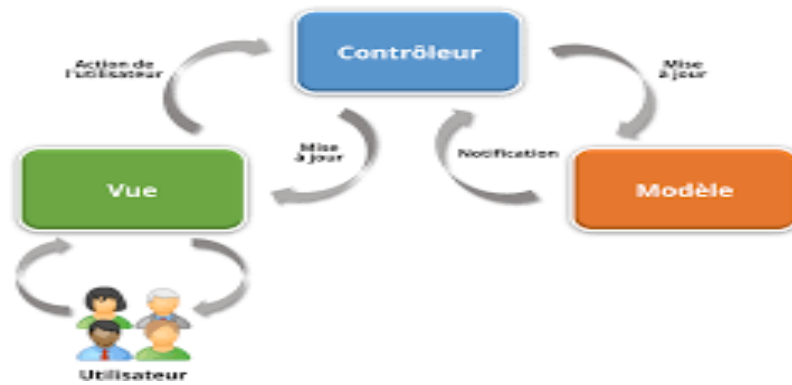


Figure 20: Architecture MVC

2. Pourquoi choisir l'architecture MVC

L'architecture « MVC » offre un moyen organisé de séparer le logique métier du code frontal et de fournir une conception propre et efficace en séparant les données de la vue et du contrôleur.

Conclusion

Au cours de ce chapitre, nous avons préparé le plan de travail et une spécification des besoins fonctionnels et non fonctionnels ainsi qu'une identification des acteurs. Par la suite, nous avons présenté le backlog et le diagramme des cas d'utilisation général et nous avons planifié et organisé le temps consacré à la réalisation du projet en identifiant les sprints. De plus, une présentation de l'environnement matériel et les logiciels utilisés au cours de la réalisation du projet est faite. Enfin, une description de l'architecture générale de la plateforme est donnée. Dans le prochain chapitre, nous présentons le premier sprint.